Guess Market – Exercise 4 (Web client)

Submitters

Name	ID	Email
Shady Aboarya	325556835	shadyaburaya04@gmail.com
Abed Alqader Wattad	213726094	shadyaburaya04@gmail.com

GitHub repository: https://github.com/Shady-ab/GuessMarket-EX4 (the server it works with: https://github.com/Shady-ab/GuessMarket-EX3)

How to run the web client

1. Start Tomcat with `GuessMarket.war` from Exercise 3, exactly as before (nothing changes on the server side). The server is expected at `http://localhost:8080/GuessMarket`.
2. Double-click `GuessMarketWeb\run.bat`. It needs only Node.js (no `npm install` – the client has no npm dependencies). A console window opens and stays open while the web client is used; the browser opens automatically.
3. If the browser did not open, go to:

http://localhost:3000

To log in as two different users at the same time, use two different browsers (for example Chrome and Edge) or a normal window and a private/incognito window. Tabs of the same browser share one session cookie, so they are the same user.

Optional: if port 3000 is taken, run `set GM_WEB_PORT=3001` and then `run.bat` from the same console, and open `http://localhost:3001`. A server on another address can be set with `GM_SERVER` (default `http://localhost:8080`).

What the web client supports

- **Login** – user name only, exactly as in the JavaFX client. A name that is currently logged in is rejected with the server's message and the user can try again. Reloading the page keeps the user logged in.
- **Events screen** – the same layout as the JavaFX client: on the left the filter line (method, status, commission, only my MM events) and the table of all events uploaded by the JavaFX clients; on the right "Event details and trade": details, one box per option (order book with bids/asks, or LMSR price bar), participations and trade history, and the actions Participate / Trade, Open and Close (market maker only). The trade dialogs show the market data, the shares the user holds and, for LMSR, the estimated cost and commission.
- **User screen** – name, balance, blocked state, deposit; my events and my involvement in the selected one; the account transactions (one line per action, including commissions received); all other users with balance, market maker and online state. As required, there is no file upload in the web client.
- All data is refreshed by pull every 0.5 seconds, so actions of JavaFX users appear in the browser and vice versa. The chat bonus of Exercise 3 is not part of the web client (as allowed).

- The page adapts to the window size; on a narrow window the panels are shown one under the other.

Technical design

- Plain HTML, CSS and JavaScript (no framework, no build step), in `GuessMarketWeb\public`.
- `server.js` is a small Node.js server that uses only built-in modules. It serves the web pages and forwards every request under `/GuessMarket/` to the existing Tomcat server unchanged. This way the browser sees one origin, the session cookie works as usual, and the Tomcat server and its setup are not changed at all (no CORS configuration was needed).
- The web client uses exactly the same endpoints, parameters and JSON as the JavaFX client: `/login`, `/logout`, `/events`, `/event`, `/event/open`, `/event/close`, `/event/buy`, `/event/order`, `/me`, `/users`, `/deposit`.
- Polling is done per data type (events, selected event, my account, users); a poll is skipped while the previous one is still running, the screen is redrawn only when the data changed, and the ledger is fetched as a delta (only new lines).
- All text from the server is HTML-escaped before it is shown (user and event names come from other users).

File	Role
`run.bat`	Checks that Node.js exists, starts `server.js` and opens the browser.
`server.js`	Static file server and proxy to the Guess Market server.
`public/index.html`	Page structure: login, top bar, Events and My account screens, dialog area.
`public/app.js`	API calls, polling, rendering of the screens, trade / close dialogs, deposit.
`public/app.css`	Look of the client (matching the JavaFX client) and the layout for small screens.

Working with AI tools

Which tool did we choose, and why? Did we try others?

We worked with **Cursor** (an editor with a built-in AI agent). The agent can read the whole project, so it could see the server code and the JSON objects of Exercise 3 and build the web client against the real API instead of guessing it. It can also run commands and open the page in a browser tab, so it tested its own work. Earlier in the course (Exercise 1) we used ChatGPT, where code had to be copied back and forth by hand; Cursor was much more convenient for a project with many files.

Where did the AI not "deliver" and we had to step in?

- **Understanding the requirements.** The AI writes code well, but we had to point out the constraints that are easy to miss: the web client must not change the Tomcat server or how it is started, the checker may only be asked to have Node.js, and the web client does not need file upload or the chat. Because of the first constraint we chose the "Node proxy" design instead of putting the pages into the WAR.
- **Testing with several users.** The AI tested the flows itself, but checking two users at the same time (JavaFX and browser together, a name that is already logged in, a session that ends) needed us to

think about the scenarios. This is how we noticed that two tabs of the same browser share the session cookie, which we documented above.
- **Small mistakes in test scripts.** A PowerShell test script written by the AI read an event id wrongly, so the following test steps failed; the AI found and fixed it after looking at the output.

Was there a bug or requirement the AI could not solve?

No requirement of this exercise remained unsolved. The problems we met were solved after one or two more rounds of prompts and testing. In Exercise 3 (JavaFX) there was one case where an automatic test tool of the AI sent keystrokes to the wrong window; we stopped that approach and the tests were changed to work without any keyboard or mouse automation.

How much of the work was done by the AI alone?

Almost all of the web client code (we estimate about 90–95%) was written by the AI from our prompts. Our part was defining what to build and the constraints, deciding on the design, reviewing the screens against the JavaFX client, and testing it ourselves with the JavaFX client and the browser side by side. We did need to understand what was happening (sessions and cookies, the proxy, polling) to give the right instructions and to check the result.

How long did it take, and how long would it take without AI?

With the AI the web client took a few hours including testing and this document. Without AI, and without prior front-end knowledge, we estimate it would have taken us two to three weeks: learning HTML, CSS and JavaScript, the browser's fetch API and cookies, and building and debugging the screens.

Did we have prior front-end knowledge?

Only basic knowledge of HTML. It helped a little in reading the generated code and in describing what we wanted to see on the screen, but the exercise could have been done without it.

Was the AI-driven style fun compared with the rest of the course?

Yes. It was fast and it was satisfying to see a full working web client so quickly. On the other hand we felt less "ownership" of the code than in the exercises we wrote ourselves, and we had to make sure we understand what the AI did before submitting it.